

# Django Vue Lab

## Занятие 4. Vue 3

Черепанов И.Н. Филиппьев В.А.

# Содержание

- Vue 3: назначение, место во фронтенде
- Создание приложения и подключение к странице
- Шаблоны: интерполяция, условия, циклы, события
- Связывание атрибутов, классов, стилей и форм
- watch и реактивность: ref, reactive, computed
- Компоненты: props, emit, slots
- Composition API и `<script setup>`
- Работа с формами и HTTP API
- Vue Router, Vite
- Интеграция с Django

## Vue 3

Vue – JavaScript-фреймворк для создания пользовательских интерфейсов.

- декларативное описание интерфейса
- реактивное обновление DOM
- компонентный подход
- можно подключить к одной странице или собрать полноценное SPA
- хорошо подходит для интерфейсов поверх HTTP API

## Когда нужен Vue

Vue полезен, когда на странице много состояния и интерактивности.

- фильтры, поиск, пагинация без перезагрузки страницы
- формы с динамическими полями
- личные кабинеты и административные панели
- интерфейсы, которые часто обращаются к API
- SPA с клиентской маршрутизацией

Если страница почти статическая, обычного Django template часто достаточно.

# Подключение через CDN

Самый простой способ попробовать Vue – подключить его прямо в HTML.

```
<div id="app">
  {{ message }}
</div>

<script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
<script>
  const { createApp } = Vue;

  createApp({
    data() {
      return {
        message: "Привет, Vue!",
      };
    }
  })
```

## Подключение через CDN (ii)

```
    },  
    }).mount("#app");  
</script>
```

# Приложение Vue

Приложение создаётся через `createApp()` и монтируется в конкретный элемент страницы.

```
import { createApp } from "vue";  
import App from "./App.vue";  
  
createApp(App).mount("#app");
```

```
<body>  
  <div id="app"></div>  
</body>
```

Vue управляет только тем DOM, который находится внутри `#app`.

## npm и Vite

npm – менеджер пакетов JavaScript. Через него устанавливают Vue, Vite и другие библиотеки.

Vite – инструмент для разработки и сборки фронтенда.

```
npm create vue@latest frontend  
cd frontend  
npm install  
npm run dev
```

- `npm install` – установить зависимости из `package.json`
- `npm run dev` – запустить dev-сервер Vite
- `npm run build` – собрать production-версию
- `npm run preview` – проверить собранный проект локально



## Vite в разработке



В разработке Vite нужен как отдельный сервер: он быстро пересобирает Vue и обновляет страницу.

## Vite в production

`npm run build`



собранные файлы  
`dist/assets/*.js`  
`dist/assets/*.css`



Django / Nginx  
отдаёт статику и API

В production Vite-сервер не запускают: браузер получает уже скомпилированный JavaScript и CSS.

# Single File Component

```
<script setup>
const title = "Список статей";
</script>

<template>
  <h1>{{ title }}</h1>
</template>

<style scoped>
h1 {
  color: #42b883;
}
</style>
```

- script – логика компонента
- template – HTML-шаблон
- style – стили компонента

# Options API и Composition API

Во Vue 3 можно писать компоненты двумя стилями.

```
<!-- Options API -->
<script>
export default {
  data() {
    return { count: 0 };
  },
  methods: {
    increment() {
      this.count = this.count + 1;
    },
  },
};
</script>
```

## Options API и Composition API (ii)

```
<!-- Composition API -->
<script setup>
import { ref } from "vue";

const count = ref(0);

function increment() {
  count.value = count.value + 1;
}
</script>
```

Composition API удобнее для сложной логики и чаще используется в новых проектах Vue 3.

# Интерполяция

В шаблоне можно выводить значения из состояния компонента.

```
<script setup>
const user = {
  name: "Анна",
  email: "anna@example.com",
};
</script>

<template>
  <h1>{{ user.name }}</h1>
  <p>{{ user.email }}</p>
</template>
```

Внутри `{{ ... }}` пишут JavaScript-выражения, но не сложную бизнес-логику.

# Шаблон Vue

template похож на обычный HTML, но Vue добавляет к нему директивы.

```
<template>
  <h1>{{ title }}</h1>
  <p v-if="isPublished">Опубликовано</p>
  <a :href="articleUrl">Открыть</a>
  <button @click="save">Сохранить</button>
</template>
```

- {{ ... }} – вывести значение
- v-if, v-for, v-model – директивы
- :href – короткая форма v-bind:href
- @click – короткая форма v-on:click

## Выражения в шаблонах

```
<template>
  <p>{{ user.firstName + " " + user.lastName }}</p>
  <p>{{ isAdmin ? "Администратор" : "Пользователь" }}</p>
  <p>{{ title.toUpperCase() }}</p>
</template>
```

**Не стоит** писать в шаблоне длинные вычисления.

```
<script setup>
  const fullName = `${user.firstName} ${user.lastName}`;
</script>

<template>
  <p>{{ fullName }}</p>
</template>
```



# Директивы

Директива – специальный атрибут Vue, который влияет на элемент.

```
<template>
  <p v-if="isVisible">Текст</p>
  <li v-for="item in items" :key="item.id">{{ item.title }}</li>
  <input v-model="query">
  <button @click="search">Найти</button>
</template>
```

Обычно директивы связывают DOM с состоянием компонента.

## Условия: v-if

v-if управляет тем, есть ли элемент в DOM.

```
<template>
  <p v-if="isLoading">Загрузка...</p>
  <p v-else-if="error">Ошибка: {{ error }}</p>
  <ArticleList v-else :articles="articles" />
</template>
```

v-show не удаляет элемент из DOM, а переключает CSS-свойство display.

```
<p v-show="isOpen">Дополнительная информация</p>
```

## Циклы: v-for

v-for выводит список элементов.

```
<template>
  <ul>
    <li v-for="article in articles" :key="article.id">
      {{ article.title }}
    </li>
  </ul>
</template>
```

key помогает Vue правильно обновлять список при добавлении, удалении и сортировке элементов.

## Атрибуты: v-bind

v-bind связывает HTML-атрибут со значением из JavaScript.

```
<template>
  <a :href="article.url" :title="article.title">
    {{ article.title }}
  </a>

  <button :disabled="isSaving">
    Сохранить
  </button>
</template>
```

Короткая форма :href эквивалентна v-bind:href.

## Классы и стили

Vue умеет удобно связывать class и style с состоянием.

```
<script setup>
const isActive = true;
const isError = false;
const progress = 70;
</script>

<template>
  <button :class="{ active: isActive, error: isError }">
    Отправить
  </button>

  <div :style="{ width: progress + '%' }">
    {{ progress }}%
  </div>
</template>
```

## Классы и стили (ii)

Так шаблон остаётся декларативным: классы зависят от данных, а не выставляются вручную через DOM.

## События: v-on

v-on назначает обработчик события.

```
<script setup>
function saveArticle() {
  console.log("Сохранить");
}
</script>

<template>
  <button @click="saveArticle">Сохранить</button>
  <form @submit.prevent="saveArticle">
    <input name="title">
  </form>
</template>
```

Короткая форма @click эквивалентна v-on:click.

## Формы: v-model

v-model связывает поле формы с состоянием компонента.

```
<script setup>
import { ref } from "vue";

const title = ref("");
</script>

<template>
  <input v-model="title" placeholder="Название статьи">
  <p>Значение: {{ title }}</p>
</template>
```

Когда пользователь печатает в input, значение title меняется автоматически.



# Модификаторы v-model

```
<template>
  <input v-model.trim="username">
  <input v-model.number="age" type="number">
  <input v-model.lazy="email">
</template>
```

- .trim – убирает пробелы по краям строки
- .number – приводит значение к числу
- .lazy – обновляет значение после change, а не после каждого ввода

## Зачем нужна реактивность

В обычном JavaScript изменение переменной не меняет HTML автоматически.

```
let count = 0;  
count = count + 1;
```

Во Vue состояние компонента связано с шаблоном.

```
<template>  
  <p>{{ count }}</p>  
</template>
```

Когда реактивное значение меняется, Vue сам обновляет нужную часть DOM.

## Реактивность: ref

`ref()` создаёт реактивное значение. Когда оно меняется, Vue обновляет шаблон.

```
<script setup>
import { ref } from "vue";

const count = ref(0);

function increment() {
  count.value = count.value + 1;
}
</script>

<template>
  <button @click="increment">
    Нажато: {{ count }}
  </button>
</template>
```

## Реактивность: ref (ii)

```
</button>  
</template>
```

В JavaScript используется `.value`, в шаблоне Vue раскрывает `ref` автоматически.

## Реактивность: reactive

`reactive()` используют для объектов с несколькими полями.

```
<script setup>
import { reactive } from "vue";

const form = reactive({
  title: "",
  text: "",
});

function clearForm() {
  form.title = "";
  form.text = "";
}
</script>
```

## Реактивность: reactive (ii)

ref удобен для отдельных значений, reactive – для связанных полей объекта.

## computed

`computed()` описывает значение, которое вычисляется из реактивного состояния.

```
<script setup>
import { computed, ref } from "vue";

const query = ref("");
const articles = ref([
  { id: 1, title: "Django" },
  { id: 2, title: "Vue" },
]);

const filteredArticles = computed(() => {
  return articles.value.filter(article =>
    article.title.toLowerCase().includes(query.value.toLowerCase())
  );
});
```

## computed (ii)

```
});  
</script>
```



## watch

`watch()` запускает код при изменении реактивного значения.

```
<script setup>
import { ref, watch } from "vue";

const page = ref(1);

watch(page, async newPage => {
  console.log("Загрузить страницу", newPage);
});
</script>
```

`computed` нужен для вычислений, `watch` – для побочных эффектов: запросов, записи в `localStorage`, аналитики.

# Компоненты

Компонент – независимая часть интерфейса со своим шаблоном и логикой.

```
<!-- ArticleCard.vue -->
<script setup>
defineProps({
  article: Object,
});
</script>

<template>
  <article>
    <h2>{{ article.title }}</h2>
    <p>{{ article.preview }}</p>
  </article>
</template>
```

## Компоненты (ii)

Компоненты помогают разделять интерфейс на понятные части.

## Использование компонента

```
<script setup>
import ArticleCard from "./ArticleCard.vue";

const articles = [
  { id: 1, title: "Django", preview: "ORM и view" },
  { id: 2, title: "Vue", preview: "Компоненты" },
];
</script>

<template>
  <ArticleCard
    v-for="article in articles"
    :key="article.id"
    :article="article"
  />
</template>
```

# Props

props – входные данные компонента от родителя к дочернему компоненту.

```
<script setup>
const props = defineProps({
  title: {
    type: String,
    required: true,
  },
  isPublished: {
    type: Boolean,
    default: false,
  },
});
</script>

<template>
```

## Props (ii)

```
<h2>{{ props.title }}</h2>  
</template>
```

Props не нужно изменять внутри дочернего компонента.

# emit

emit – событие от дочернего компонента к родителю.

```
<!-- DeleteButton.vue -->
<script setup>
const emit = defineEmits(["delete"]);
</script>

<template>
  <button @click="emit('delete')">Удалить</button>
</template>
```

```
<DeleteButton @delete="deleteArticle(article.id)" />
```

## Props и события: схема

Данные обычно идут сверху вниз, а события – снизу вверх.

Родительский компонент

state: article

метод: deleteArticle(id)

props: :article="article"

|

v

Дочерний компонент

defineProps({ article: Object })

emit("delete", article.id)

^

|

событие: @delete="deleteArticle"



## Props и события: схема (ii)

### Родительский компонент

- props передают данные в дочерний компонент
- emit сообщает родителю, что что-то произошло
- дочерний компонент не должен напрямую менять состояние родителя

# Slots

slot позволяет передать в компонент фрагмент шаблона.

```
<!-- BaseCard.vue -->
<template>
  <section class="card">
    <slot />
  </section>
</template>
```

```
<BaseCard>
  <h2>Новая статья</h2>
  <p>Форма создания статьи</p>
</BaseCard>
```

## Жизненный цикл

Компонент создаётся, монтируется в DOM, обновляется и удаляется.

```
<script setup>
import { onMounted, onUnmounted } from "vue";

onMounted(() => {
  console.log("Компонент появился на странице");
});

onUnmounted(() => {
  console.log("Компонент удалён");
});
</script>
```

`onMounted` часто используют для первой загрузки данных с сервера.

## Загрузка данных

```
<script setup>
import { onMounted, ref } from "vue";

const articles = ref([]);
const isLoading = ref(false);

async function loadArticles() {
  isLoading.value = true;
  const response = await fetch("/api/articles/");
  articles.value = await response.json();
  isLoading.value = false;
}

onMounted(loadArticles);
</script>
```

Состояния `isLoading` и `error` лучше хранить явно.

## Отправка формы

```
<script setup>
import { reactive } from "vue";

const form = reactive({
  title: "",
  text: "",
});

async function submitForm() {
  await fetch("/api/articles/", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(form),
  });
}
</script>
```

## Ошибки API

```
async function request(url, options) {  
  const response = await fetch(url, options);  
  
  if (!response.ok) {  
    const data = await response.json();  
    throw new Error(data.detail || "Ошибка запроса");  
  }  
  
  return response.json();  
}
```

Клиентский код должен отдельно обрабатывать статусы 400, 403, 404, 500.

# CSRF и Django

Для POST, PUT, PATCH, DELETE Django обычно требует CSRF-токен.

```
function getCookie(name) {  
  return document.cookie  
    .split("; ")  
    .find(row => row.startsWith(name + "="))  
    ?.split("=")[1];  
}  
  
await fetch("/api/articles/", {  
  method: "POST",  
  headers: {  
    "Content-Type": "application/json",  
    "X-CSRFToken": getCookie("csrftoken"),  
  },  
  body: JSON.stringify(form),  
});
```

# Vue Router

Vue Router добавляет клиентскую маршрутизацию.

```
import { createRouter, createWebHistory } from "vue-router";
import ArticleList from "./pages/ArticleList.vue";
import ArticleDetail from "./pages/ArticleDetail.vue";

const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: "/articles", component: ArticleList },
    { path: "/articles/:id", component: ArticleDetail },
  ],
});

export default router;
```



## router-link и router-view

```
<template>
  <nav>
    <RouterLink to="/articles">Статьи</RouterLink>
    <RouterLink to="/profile">Профиль</RouterLink>
  </nav>

  <RouterView />
</template>
```

- RouterLink рисует ссылку без полной перезагрузки страницы
- RouterView показывает компонент текущего маршрута

## Параметры маршрута

```
<script setup>
import { onMounted, ref } from "vue";
import { useRoute } from "vue-router";

const route = useRoute();
const article = ref(null);

onMounted(async () => {
  const response = await fetch(`/api/articles/
${route.params.id}/`);
  article.value = await response.json();
});
</script>
```

Маршрут `/articles/42` даст значение `route.params.id === "42"`.

# Vite

Vite – инструмент разработки и сборки фронтенда.

```
npm create vue@latest frontend  
cd frontend  
npm install  
npm run dev
```

- быстрый dev-сервер
- сборка production-файлов
- поддержка .vue компонентов
- удобная настройка прокси к Django API

## Прокси к Django API

Во время разработки Vue может работать на `localhost:5173`, а Django – на `localhost:8000`.

```
// vite.config.js
export default {
  server: {
    proxy: {
      "/api": "http://localhost:8000",
    },
  },
};
```

После этого `fetch("/api/articles/")` из Vue попадёт в Django.

# Интеграция с Django

Есть два распространённых варианта.

## Django templates + Vue widgets

- Django отдаёт HTML-страницы
- Vue монтируется на отдельные интерактивные блоки
- удобно для постепенного внедрения

## Django API + Vue SPA

- Django отдаёт JSON API
- Vue отвечает за весь интерфейс
- удобно для сложных клиентских приложений

# Django template и Vue

Синтаксис `{{ ... }}` есть и в Django templates, и во Vue.

```
<div id="app">
  [[ message ]]
</div>

<script>
  Vue.createApp({
    delimiters: ["[", "]", "],",
    data() {
      return { message: "Привет" };
    },
  }).mount("#app");
</script>
```

Для виджетов внутри Django-шаблонов удобно заменить delimiters.

## Сборка production

```
npm run build
```

Vite создаёт статические файлы в dist.

```
frontend/dist/assets/index-abc123.js  
frontend/dist/assets/index-def456.css
```

Их можно отдавать через Django static files или через отдельный веб-сервер.

## Частые ошибки

- забыли `.value` у `ref` в JavaScript
- изменяют `props` внутри дочернего компонента
- используют индекс массива как `key` при изменяемом списке
- не обрабатывают состояние загрузки и ошибки API
- отправляют POST-запрос в Django без CSRF-токена
- смешивают ответственность: компонент одновременно делает всё



## Практика

Создать Vue-интерфейс для списка статей.

- загрузить статьи из `/api/articles/`
- показать состояние загрузки
- вывести список через `v-for`
- добавить поиск по названию через `computed`
- открыть детальную страницу статьи через `Vue Router`
- добавить форму создания статьи с `v-model`
- отправить данные в Django API через `fetch`

## Итоги

- Vue описывает интерфейс декларативно: состояние меняется, DOM обновляется автоматически
- Основной строительный блок – компонент
- В Vue 3 чаще используют Composition API и `<script setup>`
- `ref`, `reactive`, `computed`, `watch` управляют реактивностью
- С Django Vue удобно связывать через JSON API
- Для реального проекта обычно используют Vite и Vue Router